Lab

Collections and generics

Lab: Collections and generics

Objective

In this lab, you will practice using **ArrayList**, **Queues**, and **HashMap**.

Part 1 — Use an ArrayList<Shape>

1. Open the bouncing shape exercise project which you worked on before.
2. Change the array of Shapes (in the **Game** class) to **ArrayList<Shape>**.

   Tip: You'll need to use the **ArrayList**'s **add()** method in the constructor's code to add the three or more instances.
3. Test your code to make sure it works.

Part 2 — Using a Queue

In this part, you will create a Queue to hold many Shopping baskets for processing.

1. Back in the **labs** project add a new class called *Program* with main() method, in a package **lab5**.
2. Add a new class called **ShoppingBasketItem** to the **lab5** package with following fields:

   String **productName;**

   int **quantity;**

   double **price;**
3. Add a constructor for the **ShoppingBasketItem** class to set the above fields and then add a method called **getDetails()** to return a String with the values of the fields.
4. Create another class called **ShoppingBasket**.
5. Create a **ArrayList<ShoppingBasketItem>** field in the ShoppingBasket class.

6. Add a new method to **ShoppingBasket** called **add**() like

   void add(**ShoppingBasketItem item**){

   }

   This method adds the **item** to the ArrayList<ShoppingBasketItem> field.
7. In the **Program** class, create a **Queue** of **ShoppingBasket**s called **baskets**.
8. Make sure each **ShoppingBasket** has one or more **ShoppingBasketItem** added.

   Since we are going to use this queue in main(), it needs to be **static**.
9. Create a **static** method in the Program class called **processBaskets()**.

   Call this method from **main()**.
10. Write code in the **processBaskets ()** method to display the **ShoppingBaskets** using the Java's Queue methods. This method simulates processing of shopping baskets for payment and shipping.
11. Call the **processBaskets()** method in main() to test your code.

Part 3 — Using HashMap<K,V>

Scenario: A zoo has a number of current animals and is expecting new arrivals soon. They wish to keep track of which animal types they have and record the count of each animal type.

You will create HashMap<String, int>. The key to String is to store animal type and a value of Integer for the count of occurrences (Lion 3, Zebra 2, etc.)

Step-by-step

1. Comment out the code in **main()** to get ready for this exercise.
2. Create a class called Zoo with a default constructor.
3. Create an instance of Zoo in main(). This will kick start the constructor where all your code will be placed.
4. Declared and initialise the following fields.

   The following String arrays contain the names of the existing animals and the new animals we will add to our zoo. The HashMap will keep track of the animals and their count.

```
HashMap<String, Integer> animalMap = null;

String[ ] originalAnimals = {"Zebra", "Lion", "Buffalo"};

String[ ] newAnimals = {"Zebra", "Gazelle", "Buffalo", "Zebra"};
```

5. Instantiate the animalMap (new it) in the class constructor.
6. Create a method in Zoo as:
   void addAnimals(String[ ] animals**)**
7. Call `addAnimals()` twice from the constructor and pass each of the two arrays (***originalAnimals*** *and* ***newAnimals***) to this method.

   addAnimals() should iterate over the `String[]` (method parameter) and add each String to the **animalMap**.
   If the animal is in the HashMap already, its count will be increased by one, otherwise its count will be set to **1**.

   Tip: Use HashMap's `containsKey()` method to see if animal exists in the HashMap object.

   You will also use the HashMap's `put()` method to put an entry back in the collection and use the HashMap's `get()` method to get the count of an animal type.

   Tip: Review the code on the slides.

8. Create a new method called **`displayAnimalData`**(). Call this method from the constructor.
   Tip: Best use an enhanced for loop.
   This method is going to display key/value pairs in two columns like this:

   Zebra **3**
   Lion **2**
   Buffalo **5**

9. Run and test your code.

Learn. To Change.

QA.com